DLDCA Inlab: One Copy Is Enough

Week 10

November 6, 2026

"There can be only one."

— *Highlander*, on the copies of a cache line in an exclusive hierarchy

Welcome back to Simulator World!

This week uses the same ChampSim, traces and makefile as Week 9. Tasks 1 and 2 repeat last week's replacement policies. Task 3 introduces inclusion policies: you make the LLC exclusive of the L2C. Keep the Week 9 PDF open, if you want.

Necessary tools

`g++`, `make` and `python3` build ChampSim; `xz` reads the traces. Check with `g++ --version`, `make --version`, `python3 --version` and `xz --version`.

Traces and where to run

We use four traces: `soplex`, `bwaves`, `mcf` and `gcc`. The lab `makefile` finds a trace by name: `trace=soplex` picks the file matching `*soplex*.xz` in `CS231_TRACE_DIR`.

Lab machines. Nothing to set up. The traces are in `/opt/CS231_Traces`.

Running locally. Do this once:

1. **Download ChampSim:** get the lab folder, which contains `Champsim/`. Or clone the ChampSim repo at tag `2022-01` (link in the Week 9 PDF).
2. **Download the traces:** the four files below (to start, get at least `mcf` and `bwaves`), from the DPC-3 page (`https://dpc3.compas.cs.stonybrook.edu/champsim-traces/speccpu/`), into one folder.
3. **Point the makefile at them:** `export CS231_TRACE_DIR=[that folder]`. This line is required.

```
603.bwaves_s-891B.champsimtrace.xz   # get these two first
605.mcf_s-994B.champsimtrace.xz
602.gcc_s-2226B.champsimtrace.xz      # then these; if the exact name is missing,
soplex_66B.trace.xz                   # take a similarly named file
```

Recap: running ChampSim

`Champsim/` is the same copy as last week. Cache sizes in its `champsim_config.json`, with 64-byte lines:

cache	sets	ways	size	hit latency (cycles)
L1D	64	12	48 KB	5
L2C	1024	8	512 KB	10
LLC	2048	16	2 MB	20

Build and run ChampSim by hand

From inside `Champsim/`: configure (reads the JSON, writes the `Makefile`), compile (binary goes to `bin/`), then run with a trace and two instruction counts.

```
$ make clean                       # only if you built before
$ ./config.sh champsim_config.json
$ make
$ ./bin/champsim --warmup_instructions 1000000 \
                 --simulation_instructions 10000000 \
                 $CS231_TRACE_DIR/soplex_66B.trace.xz
```

Changed the JSON or added a policy? Rerun `./config.sh` and `make`. `make` alone does not read the JSON. On the lab machines, use `/opt/CS231_Traces` in place of `$CS231_TRACE_DIR`.

A **quick run** (1M warm-up + 10M simulated, under a minute) checks that your code builds and behaves. A **long run** (20M + 25M, a few minutes per trace) gives the numbers worth comparing. Only the simulated instructions count after `Region of Interest Statistics`.

The three numbers that matter: IPC, L2C MPKI and LLC MPKI

- **IPC** (instructions per cycle; higher is better). Read it from the line `CPU 0 cumulative IPC: [ipc]`.
- **MPKI** (misses per 1000 instructions; lower is better) of a cache. ChampSim does not print it. Compute it from the `MISS` of that cache's `TOTAL` line (`cpu0_L2C TOTAL` or `LLC TOTAL`) and the `--simulation_instructions` you passed:

$$\text{MPKI} = \frac{\text{TOTAL MISS}}{\text{simulation instructions}/1000}$$

  For example, 356 121 L2C misses in 10 000 000 instructions give an L2C MPKI of 35.6121.

The lab makefile prints all three for you, and `make check` compares exactly these three numbers with ours. New this week: the LLC MPKI is checked too.

Inclusion policies

A line can have a copy in more than one level at once. An *inclusion policy* says which combinations are allowed. Let C_{L2} be the set of lines held in the L2C and C_{LLC} the set held in the LLC.

policy	rule	meaning
inclusive	$C_{L2} \subseteq C_{LLC}$	every line in the L2C is also in the LLC
exclusive	$C_{L2} \cap C_{LLC} = \varnothing$	no line is in both
NINE	neither of the two	non-inclusive, non-exclusive: copies are allowed but not required

Our ChampSim is NINE. Every level keeps a copy of every line it fetches, and an eviction never removes the copies above it. In Task 3 you make the LLC exclusive of the L2C.

Building and testing

The lab makefile (run it from the lab folder, not from `Champsim/`)

`make task1`	build your `task1/mylru.cc` at the L2C; quick run on `soplex`
`make task2`	the same for `task2/ip_based.cc`
`make task3`	build ChampSim with `task3.patch` and your `task3/cache.cc`; quick run on `soplex`
`make lru`	ChampSim's own `lru` at the L2C and the given `cache.cc`: the baseline
`... trace=mcf`	use another trace: `soplex`, `bwaves`, `mcf` or `gcc`
`... long=1`	long run (20M + 25M) instead of quick
`make check`	check every task against our numbers
`make check-task3`	check one task (also `check-task1`, `check-task2`)
`make clean`	delete everything that was built

Every run ends with the numbers you need: the `CPU 0 cumulative IPC` line and the `cpu0_L2C MPKI` and `LLC MPKI` lines.

For Tasks 1 and 2, `make taskN` copies your file into `Champsim/replacement/[policy]/`, writes `Champsim/[policy].json` with that policy at the L2C, builds `Champsim/bin/[policy]` and runs it, as last week. For Task 3, `make task3` copies `Champsim/` to a hidden folder `.task3_champsim/`, applies `task3/task3.patch`, puts your `task3/cache.cc` in place of the original and builds it. The copy is made again on every build, so your edits never reach the other tasks. Edit only the files in the task folders.

`make check` runs each task for 1M + 5M instructions and compares the result with our solution's. **The IPC, the L2C MPKI and the LLC MPKI must all equal the values in `taskN/expected_output.txt` exactly**; if any differs, the check prints your values and ours and reports FAIL. ChampSim is deterministic, so a correct solution gives exactly our numbers.

With the unchanged `task3/cache.cc`, every trace fails:

```
$ make check-task3
Building .task3_champsim/bin/champsim with task3.patch and task3/cache.cc
[task3] soplex  | FAIL (differs from the reference)
    got:      IPC 0.334436, L2C MPKI 34.0892, LLC MPKI 27.6468
    expected: IPC 0.356154, L2C MPKI 34.0892, LLC MPKI 58.9158
[task3] mcf     | FAIL (differs from the reference)
    got:      IPC 0.647025, L2C MPKI 14.1932, LLC MPKI 7.0546
    expected: IPC 0.670564, L2C MPKI 14.1938, LLC MPKI 20.5920
...
```

The traces run at the same time, so the lines can come in any order. If a test keeps failing for no reason you can see, run `make clean`.

Structure of submission/templates

Edit only the files marked `(TODO)`.

Submission format:

```
your-roll-no/
|- labexam/
    |- sinestro.cc (TODO)
    |- expected_output.txt
|- task1/
    |- mylru.cc (TODO)
    |- expected_output.txt
|- task2/
    |- ip_based.cc (TODO)
    |- expected_output.txt
|- task3/
    |- cache.cc (TODO)
    |- task3.patch
    |- apply_patch.sh
    |- expected_output.txt
```

The folder structure of the expected submission for this lab is given above. We expect this folder to be compressed into a tarball with the naming convention of `your-roll-no.tar.gz` in lowercase. The command given below can be used to do the same

```
$ ./build_archive.sh
Enter Student Roll Number: your-roll-no
Packaging assignment files...
  [+] Copied: ...
Successfully created your-roll-no.tar.gz
```

Tasks

Lab Exam: In Brightest Way

Somewhere in Sector 2814, Hal and John were killing time at a way station, arguing about caches.

Hal: LRU trusts every newcomer: a new line goes in as **most recently used**, and the **least recently used** line gets **evicted**. The Corps trusted a newcomer like that once. Sinestro.

John: That's the way it always goes, what's wrong with it?

Hal: Ok junior, I'll show you. Take a 4-way set and a loop over 5 lines: $a_0, a_1, a_2, a_3, a_4, a_0, a_1, \ldots$ The set holds $a_0..a_3$. Then a_4 arrives and evicts a_0, the very next line we need. a_0 comes back and evicts a_1, and so on. Every access misses, forever. That's a *cache-thrashing workload.*

John: That's easy. Put the newcomers on probation, that is, insert them as the *least* recently used. Now a_4 evicts one line, say a_3, and sits in the LRU slot. When a_3 comes back, it evicts

a_4, the LRU line, and takes that slot. The newcomers fight over one way, while a_0, a_1, a_2 stay put and hit every pass: 3 hits out of 5 instead of 0.

Hal: And when the program moves on to $b_0, b_1, \ldots$? Your newcomers never get promoted.

John: Every 32nd fill goes in as most recently used, like plain LRU does every time. A new working set works its way in slowly.

Hal: ... That's clean. Spoken like an architect, Junior.

Then the ring chimed, "junior, they're playing our song" and they went back to looking for Sinestro. You should get back to the lab exam too btw.

The policy. Track two things. (1) An age per line, as in Task 1: $W - 1$ is the most recently used line and 0 the least recently used (W is `NUM_WAY`). Evict the line with age 0. (2) To know when you're on the 32nd fill, keep one fill counter for the whole cache, starting at 0 (note: count *fills*, not accesses). The policy has three parts:

1. **Eviction** (in `find_victim`):

```
return the way whose age is 0
```

2. **Hit** (in `update_replacement_state`, when `hit = 1`): move the line to the most recently used position, exactly as in Task 1.

```
a = age of (set, way)
for every way i of the set:
    if age of i > a:               # for any lines more recently used than a,
        age of i = age of i - 1    #    age them once
age of (set, way) = W - 1
```

3. **Fill** (in `update_replacement_state`, when `hit = 0`): count the fill. Every 32nd fill is inserted as the most recently used line, all others as the least recently used line.

```
fill counter = fill counter + 1
if fill counter == 32:
    fill counter = 0       # rare: ~1/32 of fills, insert at MRU ;)
    do the same as for a hit
else:
    age of (set, way) = 0  # common: ~31/32 of fills, insert at LRU :)
                           # no other age changes (we are 0)
```

A fill goes into the way that `find_victim` just returned (age 0), so inserting at age 0 changes no other age.

Ignore translation and prefetch accesses. The policy must not react to accesses of type `TRANSLATION` or `PREFETCH`: no age change and no fill count. The template already contains the code for it at the top of `update_replacement_state`. **Do not remove or change this snippet:**

```
if (type == TRANSLATION || type == PREFETCH)
    return;
```

Hit and fill are different events. Use the parameter `hit=0` or `hit=1` to tell them apart. Note that `WRITEBACK`, `LOAD` and `RFO` are all treated equally here.

Task. Edit only `labexam/sinestro.cc`. Fill in its four `TODO`s:

1. at the top of the file: declare your data structures (the ages and the fill counter) — see the way it's done in Task 1 + just add a fill counter;
2. in `initialize_replacement`: initialize them;
3. in `find_victim`: write the eviction logic;
4. in `update_replacement_state`: write the hit and fill logic.

Declare the ages and the counter per cache, with a `std::map` from `CACHE*` as in Task 1.

Build and run. This policy sits at the L2C. Use the makefile (below), or do it by hand. **By hand:** after every edit of `sinestro.cc`, do these steps in `Champsim/`:

1. Copy `labexam/sinestro.cc` to `Champsim/replacement/sinestro/sinestro.cc`.
2. In `champsim_config.json`, in the `L2C` block, change the value of `replacement` from `lru` to `sinestro`.
3. Run `./config.sh champsim_config.json`, then `make`, then `./bin/champsim` with the trace and the two instruction counts, as in the recap above.

Makefile: the lab `makefile` does all of it. Run it from the lab folder (the one with `labexam/`), not from `Champsim/`:

`make labexam`	test run (1M + 10M instructions) on `soplex`
`make labexam trace=[trace]`	test run on another trace
`make labexam long=1 trace=[trace]`	full run (20M + 25M), **used for grading**
`make labexam-check`	check against our numbers, all four traces
`make clean`	delete everything that was built

Check. `make labexam-check` builds your `sinestro.cc` and runs it on all four traces. **The IPC, the L2C MPKI and the LLC MPKI must all equal the values in `labexam/expected_output.txt` exactly**; ChampSim is deterministic, so a correct policy gives exactly our numbers. Each trace prints PASS, or FAIL with your values and ours. The check uses shorter runs than the full runs in the table below, so its numbers differ from the table; the grading uses the full runs. A correct solution looks like this (the traces run at the same time, so the lines can come in any order):

```
$ make labexam-check
Building Champsim/bin/sinestro (L2C replacement = sinestro)
[labexam] soplex  | PASS  IPC 0.351972, L2C MPKI 40.6966, LLC MPKI 27.6178
[labexam] mcf     | PASS  IPC 0.635395, L2C MPKI 21.7644, LLC MPKI 7.0450
...
```

If the unchanged template is checked, every trace reports FAIL. If a check keeps failing for no reason you can see, run `make clean` and try again.

Submit. Submit `labexam/sinestro.cc` in the submission folder given at the start of this document, packed with `./build_archive.sh`.

Reference numbers: IPC, L2C MPKI and LLC MPKI

Your full-run IPC, L2C MPKI and LLC MPKI are compared with those of our reference implementation of `sinestro` on all four traces. Our full runs on the lab machines give these numbers. The `lru` rows are ChampSim's own LRU at the L2C, for comparison.

trace	L2C policy	IPC	L2C MPKI	LLC MPKI	L1D miss latency
`soplex`	`lru`	0.323267	34.0741	27.6852	310.9
	`sinestro`	0.330552	42.7405	28.7932	307.2
`bwaves`	`lru`	0.243644	26.1799	26.1785	1400.8
	`sinestro`	0.240736	34.5926	29.4837	1400.0
`mcf`	`lru`	0.409243	16.9194	10.1910	86.3
	`sinestro`	0.403626	25.4050	10.3586	91.5
`gcc`	`lru`	0.113813	69.9909	69.9792	113.5
	`sinestro`	0.113809	70.1574	70.0143	113.5

```
$ make labexam long=1
Building Champsim/bin/sinestro (L2C replacement = sinestro)
Champsim/bin/sinestro --warmup_instructions 20000000 \
    --simulation_instructions 25000000 \
    /opt/CS231_Traces/soplex_66B.trace.xz
CPU 0 cumulative IPC: 0.330552 instructions: 25000002 cycles: 75631027
cpu0_L2C TOTAL     ACCESS:    1434585  HIT:     366072  MISS:    1068513
cpu0_L2C LOAD      ACCESS:     884474  HIT:     155780  MISS:     728694
cpu0_L2C RFO       ACCESS:     149410  HIT:      17613  MISS:     131797
cpu0_L2C WRITEBACK ACCESS:     400089  HIT:     192578  MISS:     207511
LLC TOTAL     ACCESS:    1176336  HIT:     456505  MISS:     719831
LLC LOAD      ACCESS:     728686  HIT:     173335  MISS:     555351
LLC RFO       ACCESS:     131786  HIT:      32347  MISS:      99439
LLC WRITEBACK ACCESS:     315353  HIT:     250406  MISS:      64947
cpu0_L2C MPKI: 42.7405
LLC MPKI: 28.7932
```

Task 1: Least Recently Used Replacement (Revision)

This task is Task 2 of Week 9, unchanged. If you solved it last week, copy your file in and run `make check-task1`*. The check now also compares the LLC MPKI.*

LRU evicts the line of the set that was used longest ago. It rests on the observation that a line used recently is likely to be used again soon. ChampSim ships an LRU (`replacement/lru/lru.cc`), but it is written with C++ iterators and lambdas around the `lru` field of `BLOCK`. In this task you write the same policy with a plain array and loops, and show that it is the same policy: the simulator must give exactly the same result with yours as with ChampSim's.

Step 1: An age per line. Give every line of a set an age from 0 to $W - 1$, where W is `NUM_WAY`. Each age is used by exactly one line of the set. $W - 1$ is the most recently used line, 0 the least recently used.

- **Eviction** (`find_victim`): return the way whose age is 0.
- **Hit and fill** (`update_replacement_state`): let a be the age of the line in `(set, way)`. Every line of the set whose age is greater than a loses 1. Then the line in `(set, way)` gets age $W - 1$.

The second rule keeps the ages a permutation of $0 \dots W - 1$: the lines above a move down by one and fill the gap left by a, and $W - 1$ becomes free for the line just used.

Step 2: A worked example. Take one set with $W = 4$ ways holding lines A, B, C and D. Each row shows the ages after the event on the left.

event	A	B	C	D	
start	0	1	2	3	D used last, A longest ago
hit on B	0	3	1	2	C and D were above 1, they lose 1
hit on D	0	2	1	3	only B was above 2
miss, set full					`find_victim` returns way 0 (A)
fill E into way 0	3	1	0	2	E was 0, so everyone loses 1

Step 3: The starting state. All ages start at 0, so until the set is full they are not a permutation. This is fine: while a set has an invalid way, `cache.cc` fills that way without calling `find_victim`. Each fill gives the new line age $W - 1$ and moves the other filled lines down by one. After W fills the ages are $0 \dots W - 1$, in fill order.

Step 4: Storage. The ages live in a `std::vector<uint32_t>` of size `NUM_SET` × `NUM_WAY`; the age of `(set, way)` is at index `set * NUM_WAY + way`. The template declares it as `lruBits[this]`, a map from the cache to its vector, so that two caches using the same policy do not share their ages.

Step 5: Writebacks. A writeback hit is the level above returning a dirty line, not a use by the program, so it must not change the age. The template already returns early for it, as ChampSim's `lru` does.

Task. Fill in `initialize_replacement`, `find_victim` and `update_replacement_state` in `task1/mylru.cc`. Do not read or write the `lru` field of `BLOCK`; keep the ages in `lruBits`.

Reference numbers: IPC, L2C MPKI and LLC MPKI

Long runs (`make task1 long=1 trace=[trace]`) on the lab machines give these numbers. **Your IPC, L2C MPKI and LLC MPKI must match the `mylru` rows.** The `lru` rows are ChampSim's own LRU at the L2C, for comparison.

trace	L2C policy	IPC	L2C MPKI	LLC MPKI	L1D miss latency
`soplex`	`lru`	0.323267	34.0741	27.6852	310.9
	`mylru`	0.323267	34.0741	27.6852	310.9
`bwaves`	`lru`	0.243644	26.1799	26.1785	1400.8
	`mylru`	0.243644	26.1799	26.1785	1400.8
`mcf`	`lru`	0.409243	16.9194	10.1910	86.3
	`mylru`	0.409243	16.9194	10.1910	86.3
`gcc`	`lru`	0.113813	69.9909	69.9792	113.5
	`mylru`	0.113813	69.9909	69.9792	113.5

Equal numbers to the last digit on every trace mean both policies evict the same line every time. On `soplex` the long runs of both are identical, to the last cycle:

```
$ make lru long=1
Building Champsim/bin/lru (L2C replacement = lru)
Champsim/bin/lru --warmup_instructions 20000000 \
    --simulation_instructions 25000000 \
    /opt/CS231_Traces/soplex_66B.trace.xz
CPU 0 cumulative IPC: 0.323267 instructions: 25000002 cycles: 77335507
cpu0_L2C TOTAL     ACCESS:    1434603  HIT:     582750  MISS:     851853
cpu0_L2C LOAD      ACCESS:     884477  HIT:     161932  MISS:     722545
cpu0_L2C RFO       ACCESS:     149423  HIT:      20421  MISS:     129002
cpu0_L2C WRITEBACK ACCESS:     400091  HIT:     399959  MISS:        132
LLC TOTAL     ACCESS:    1167691  HIT:     475561  MISS:     692130
LLC LOAD      ACCESS:     722539  HIT:     123466  MISS:     599073
LLC RFO       ACCESS:     129000  HIT:      36154  MISS:      92846
LLC WRITEBACK ACCESS:     315978  HIT:     315875  MISS:        103
cpu0_L2C MPKI: 34.0741
LLC MPKI: 27.6852

$ make task1 long=1
Building Champsim/bin/mylru (L2C replacement = mylru)
Champsim/bin/mylru --warmup_instructions 20000000 \
    --simulation_instructions 25000000 \
    /opt/CS231_Traces/soplex_66B.trace.xz
CPU 0 cumulative IPC: 0.323267 instructions: 25000002 cycles: 77335507
cpu0_L2C TOTAL     ACCESS:    1434603  HIT:     582750  MISS:     851853
cpu0_L2C LOAD      ACCESS:     884477  HIT:     161932  MISS:     722545
cpu0_L2C RFO       ACCESS:     149423  HIT:      20421  MISS:     129002
cpu0_L2C WRITEBACK ACCESS:     400091  HIT:     399959  MISS:        132
LLC TOTAL     ACCESS:    1167691  HIT:     475561  MISS:     692130
LLC LOAD      ACCESS:     722539  HIT:     123466  MISS:     599073
LLC RFO       ACCESS:     129000  HIT:      36154  MISS:      92846
LLC WRITEBACK ACCESS:     315978  HIT:     315875  MISS:        103
cpu0_L2C MPKI: 34.0741
LLC MPKI: 27.6852
```

Task 2: Replacement by Instruction Address (Revision)

This task is Task 3 of Week 9, unchanged. If you solved it last week, copy your file in and run `make check-task2`*. The check now also compares the LLC MPKI.*

Every line in a cache was brought in by one load or store instruction, and every instruction has an address, its `ip` (instruction pointer, also called program counter). Some instructions load data that is used again soon, such as a counter or a small table. Others load data that is used once, such as one element of a large array scanned from start to end. A policy that learns which instructions are which can keep the lines of the first kind and evict the lines of the second.

Step 1: A counter per instruction. Keep an 8-bit counter, `pcCount`, for each instruction. A high count means the lines that instruction brings in tend to be hit again.

- **Fill** (`hit = 0`): the instruction `ip` missed. Decrease its counter by 1, but not below 0.
- **Hit** (`hit = 1`): the instruction `ip` found its data. Increase its counter by 1, but not above 255.
- **Eviction** (`find_victim`): `current_set[i].ip` is the `ip` of the instruction that brought line `i` in. Evict the line whose `ip` has the lowest counter. On a tie, evict the line with the lowest way.

Hits and fills use the `ip` argument of `update_replacement_state`: the instruction making the access now. Eviction uses the `ip` stored in each line: the instruction that brought it in. Writeback hits are skipped, as in Task 1.

Step 2: Bounded storage. Addresses in a trace are 48 bits wide. A counter for every possible `ip` would need 2^{48} bytes, and hardware has a fixed table. Use only the lower 16 bits of the `ip` (`ip & 0xFFFF`) as the index into a table of $2^{16} = 65536$ counters. Two instructions whose addresses agree in the low 16 bits share a counter. The template gives you `pc_index(ip)`, which computes this index.

Step 3: A worked example. A set of $W = 4$ ways holds lines brought in by instructions with these counters:

way	`current_set[way].ip & 0xFFFF`	`pcCount`
0	`0x1a40`	7
1	`0x1a58`	0
2	`0x2210`	3
3	`0x1a58`	0

Ways 1 and 3 tie at 0, so `find_victim` returns 1. If instruction `0x1a58` now hits on a line, its counter becomes 1, and the next victim is still way 1 (count 1, lower than 3 and 7).

Task. Fill in `initialize_replacement`, `find_victim` and `update_replacement_state` in `task2/ip_based.cc`. The table is declared for you as `pcCount[this]`, a `std::vector<uint8_t>`.

Reference numbers: IPC, L2C MPKI and LLC MPKI

Long runs (`make task2 long=1 trace=[trace]`) on the lab machines give these numbers. **Your IPC, L2C MPKI and LLC MPKI must match the `ip_based` rows.** The `lru` rows are ChampSim's own LRU at the L2C, for comparison.

trace	L2C policy	IPC	L2C MPKI	LLC MPKI	L1D miss latency
`soplex`	`lru`	0.323267	34.0741	27.6852	310.9
	`ip_based`	0.333429	46.6156	25.8012	287.9
`bwaves`	`lru`	0.243644	26.1799	26.1785	1400.8
	`ip_based`	0.243644	34.9085	26.1785	1400.9
`mcf`	`lru`	0.409243	16.9194	10.1910	86.3
	`ip_based`	0.402809	25.0904	10.2190	91.3
`gcc`	`lru`	0.113813	69.9909	69.9792	113.5
	`ip_based`	0.114147	69.8671	69.6960	113.1

```
$ make task2 long=1
Building Champsim/bin/ip_based (L2C replacement = ip_based)
Champsim/bin/ip_based --warmup_instructions 20000000 \
    --simulation_instructions 25000000 \
    /opt/CS231_Traces/soplex_66B.trace.xz
CPU 0 cumulative IPC: 0.333429 instructions: 25000002 cycles: 74978442
cpu0_L2C TOTAL     ACCESS:    1434756  HIT:     269365  MISS:    1165391
cpu0_L2C LOAD      ACCESS:     884487  HIT:      98827  MISS:     785660
cpu0_L2C RFO       ACCESS:     149488  HIT:      24935  MISS:     124553
cpu0_L2C WRITEBACK ACCESS:     400169  HIT:     145300  MISS:     254869
LLC TOTAL     ACCESS:    1271210  HIT:     626181  MISS:     645029
LLC LOAD      ACCESS:     785649  HIT:     218293  MISS:     567356
LLC RFO       ACCESS:     124542  HIT:      49737  MISS:      74805
LLC WRITEBACK ACCESS:     360710  HIT:     357923  MISS:       2787
cpu0_L2C MPKI: 46.6156
LLC MPKI: 25.8012
```

Compare the `ip_based` rows with the `lru` rows in the table above: the same policy can help one program and hurt another.

Task 3: An Exclusive LLC

In the given ChampSim the hierarchy is NINE: every level keeps a copy of every line it fetches, so after a quick run of `soplex`, 8191 of the 8192 lines in the L2C are also in the LLC. The LLC spends 512 KB of its 2 MB on lines the L2C already has.

In this task you make the LLC **exclusive** of the L2C: a line is in the L2C or in the LLC, never in both. Together they then hold 2.5 MB of different lines. The LLC becomes a victim cache for the L2C. A line comes from DRAM straight into the L2C, goes down to the LLC when the L2C evicts it, and comes back up to the L2C on an LLC hit.

The patch. An exclusive LLC moves lines up and down, and a dirty line must stay dirty on the way. In the given ChampSim a packet has no dirty bit, so `task3/task3.patch` adds one (`PACKET::dirty`) and makes caches fill, write back and answer with it. With only the patch, ChampSim gives exactly the same numbers as before. The patch also makes `main.cc` print one line before the statistics, the number of valid L2C lines that also have a valid copy in the LLC:

```
L2C LINES ALSO IN LLC: 8191 of 8192
```

`make task3` applies the patch for you: it copies `Champsim/` to `.task3_champsim/`, applies the patch, puts your `task3/cache.cc` in place of `src/cache.cc`, builds and runs it. `task3/cache.cc` already contains the patch's changes to `cache.cc`. To patch another ChampSim folder by hand, run the given script on it, then edit its `src/cache.cc`:

```
$ ./task3/apply_patch.sh ~/Desktop/ChampSim
patched /home/[you]/Desktop/ChampSim
```

Your edits. You submit `task3/cache.cc`. It has an empty `// TODO:` at each place to edit, with a hint above it, and this flag at the top:

```
const bool EXCLUSIVE_LLC = true;
```

The code in `cache.cc` runs for every cache, and `NAME` tells you which one: `LLC` or `cpu0_L2C`. Guard every change with `EXCLUSIVE_LLC` (and `NAME` where it names a cache), so that with the flag set to `false` ChampSim behaves exactly as before.

Step 1: A bypass still returns its data. *Where:* `handle_fill` and `filllike_miss`.

Step 2 makes the LLC bypass its fills: the line goes up without taking a way (`way == NUM_WAY`). The given `handle_fill` sends data up only when the line took a way, so a bypass would never answer the L2C and the simulation would stop with `DEADLOCK!`. Send the data up for a bypass too, and skip the replacement update in `filllike_miss`, since a bypass has no way to update.

Step 2: A line from DRAM goes to the L2C only. *Where:* `handle_fill`, after the victim way is chosen.

An exclusive LLC holds only lines the L2C has evicted. When DRAM answers an LLC miss, the LLC bypasses the fill.

Step 3: The L2C sends every victim to the LLC. *Where:* `filllike_miss`, after `evicting_dirty` is set.

The LLC now gets its lines only from L2C evictions, so a clean victim must go down too, not only a dirty one. `evicting_dirty` decides whether the victim is sent; in the L2C, send every valid victim. The patch puts the victim's dirty bit in the packet, so a clean line arrives in the LLC clean.

Step 4: An LLC hit moves the line up. *Where:* `readlike_hit`, before the response is sent.

The L2C stores the line when the response arrives, so the LLC drops its copy. If the line is dirty, the L2C now owns the dirty data: put the dirty bit in the response (`handle_pkt`), then invalidate the LLC's block.

Check each step. Quick runs on `soplex` (`make task3`) give these numbers after each step. After step 4, `L2C LINES ALSO IN LLC` is 0. `make check-task3` prints this count for each trace too. In its shorter runs a few lines (3 on `soplex` and `mcf`) stay in both: the L2C asked for a line again while its victim was still in the LLC's write queue, so the LLC answered from the queue and then stored the victim as well. The reference solution gives the same count.

after	L2C lines also in LLC	IPC	L2C MPKI	LLC MPKI
given `cache.cc`	8191	0.317071	35.6121	28.6951
step 1	8191	0.317071	35.6121	28.6951
step 2	5800	0.383673	35.6123	30.1266
step 3	3899	0.325360	35.6121	54.0824
step 4	0	0.334986	35.6120	61.6739

Reference numbers: IPC, L2C MPKI and LLC MPKI

Long runs (`make task3 long=1 trace=[trace]`) on the lab machines give these numbers. **Your IPC, L2C MPKI and LLC MPKI must match the `exclusive LLC` rows.** The `given (NINE)` rows are the unchanged ChampSim, the same as `make lru`. The LLC MPKI doubles because every L2C victim is now an LLC write, yet the IPC goes up: `soplex` gains 6% and `mcf` 2%.

trace	cache.cc	IPC	L2C MPKI	LLC MPKI	L1D miss latency
`soplex`	given (NINE)	0.323267	34.0741	27.6852	310.9
	exclusive LLC	0.343337	34.0737	58.9574	284.2
`bwaves`	given (NINE)	0.243644	26.1799	26.1785	1400.8
	exclusive LLC	0.245052	26.1799	52.3575	1374.2
`mcf`	given (NINE)	0.409243	16.9194	10.1910	86.3
	exclusive LLC	0.418539	16.9193	26.3026	80.2
`gcc`	given (NINE)	0.113813	69.9909	69.9792	113.5
	exclusive LLC	0.113824	69.9909	139.9674	113.5

```
$ make task3 long=1
Building .task3_champsim/bin/champsim with task3.patch and task3/cache.cc
.task3_champsim/bin/champsim --warmup_instructions 20000000 \
    --simulation_instructions 25000000 \
    /opt/CS231_Traces/soplex_66B.trace.xz
L2C LINES ALSO IN LLC: 0 of 8192
CPU 0 cumulative IPC: 0.343337 instructions: 25000002 cycles: 72814766
cpu0_L2C TOTAL     ACCESS:    1434598  HIT:     582755  MISS:     851843
cpu0_L2C LOAD      ACCESS:     884472  HIT:     161938  MISS:     722534
cpu0_L2C RFO       ACCESS:     149420  HIT:      20417  MISS:     129003
cpu0_L2C WRITEBACK ACCESS:     400094  HIT:     399962  MISS:        132
LLC TOTAL     ACCESS:    1703547  HIT:     229612  MISS:    1473935
LLC LOAD      ACCESS:     722530  HIT:     175823  MISS:     546707
LLC RFO       ACCESS:     129001  HIT:      53565  MISS:      75436
LLC WRITEBACK ACCESS:     851842  HIT:        130  MISS:     851712
cpu0_L2C MPKI: 34.0737
LLC MPKI: 58.9574
```

A note on open-ended changes. A change like this one can be written in several ways, and different versions can all be correct. This one is small enough that everyone ends up with the same code, and `make check` expects our numbers exactly. In general, do not worry if your code is not identical to someone else's.